tema de archivos. Por esta razón hemos estructurado el capítulo en varias secciones: las primeras dos están relacionadas con la interfaz del usuario para los archivos y directorios, respectivamente; después incluimos un análisis detallado acerca de cómo se implementa y administra el sistema de archivos; por último, daremos algunos ejemplos de sistemas de archivos reales.

4.1 ARCHIVOS

En las siguientes páginas analizaremos los archivos desde el punto de vista del usuario; es decir, cómo se utilizan y qué propiedades tienen.

4.1.1 Nomenclatura de archivos

Los archivos son un mecanismo de abstracción. Proporcionan una manera de almacenar información en el disco y leerla después. Esto se debe hacer de tal forma que se proteja al usuario de los detalles acerca de cómo y dónde se almacena la información y cómo funcionan los discos en realidad.

Probablemente, la característica más importante de cualquier mecanismo de abstracción sea la manera en que los objetos administrados son denominados, por lo que empezaremos nuestro examen de los sistemas de archivos con el tema de la nomenclatura de los archivos. Cuando un proceso crea un archivo le proporciona un nombre. Cuando el proceso termina, el archivo continúa existiendo y puede ser utilizado por otros procesos mediante su nombre.

Las reglas exactas para denominar archivos varían un poco de un sistema a otro, pero todos los sistemas operativos actuales permiten cadenas de una a ocho letras como nombres de archivos legales. Por ende, *andrea*, *bruce* y *cathy* son posibles nombres de archivos. Con frecuencia también se permiten dígitos y caracteres especiales, por lo que nombres como *2*, *urgente!* y *Fig.2-14* son a menudo válidos también. Muchos sistemas de archivos admiten nombres de hasta 255 caracteres.

Algunos sistemas de archivos diferencian las letras mayúsculas de las minúsculas, mientras que otros no. UNIX cae en la primera categoría; MS-DOS en la segunda. Así, un sistema UNIX puede tener los siguientes nombres como tres archivos distintos: *maria*, *Maria* y *MARIA*. En MS-DOS, todos estos nombres se refieren al mismo archivo.

Tal vez sea adecuado hacer en este momento un paréntesis sobre sistemas de archivos. Windows 95 y Windows 98 utilizan el sistema de archivos de MS-DOS conocido como **FAT-16** y por ende heredan muchas de sus propiedades, como la forma en que se construyen sus nombres. Windows 98 introdujo algunas extensiones a FAT-16, lo cual condujo a **FAT-32**, pero estos dos sistemas son bastante similares. Además, Windows NT, Windows 2000, Windows XP y .WV admiten ambos sistemas de archivos FAT, que en realidad ya son obsoletos. Estos cuatro sistemas operativos basados en NT tienen un sistema de archivos nativo (NTFS) con diferentes propiedades (como los nombres de archivos en Unicode). En este capítulo, cuando hagamos referencia a los sistemas de archivos MS-DOS o FAT, estaremos hablando de FAT-16 y FAT-32 como se utilizan en Windows, a menos que se especifique lo contrario. Más adelante en este capítulo analizaremos los sistemas de archivos FAT y en el capítulo 11 examinaremos el sistema de archivos NTFS, donde analizaremos Windows Vista con detalle.

Muchos sistemas operativos aceptan nombres de archivos en dos partes, separadas por un punto, como en *prog.c*. La parte que va después del punto se conoce como la **extensión del archivo** y por lo general indica algo acerca de su naturaleza. Por ejemplo, en MS-DOS, los nombres de archivos son de 1 a 8 caracteres, más una extensión opcional de 1 a 3 caracteres. En UNIX el tamaño de la extensión (si la hay) es a elección del usuario y un archivo puede incluso tener dos o más extensiones, como en *paginainicio.html.zip*, donde *.html* indica una página Web en HTML y *.zip* indica que el archivo se ha comprimido mediante el programa *zip*. Algunas de las extensiones de archivos más comunes y sus significados se muestran en la figura 4-1.

Extensión	Significado
archivo.bak	Archivo de respaldo
archivo.c	Programa fuente en C
archivo.gif	Imagen en Formato de Intercambio de Gráficos de CompuServe
archivo.hlp	Archivo de ayuda
archivo.html	Documento en el Lenguaje de Marcación de Hipertexto de World Wide Web
archivo.jpg	Imagen fija codificada con el estándar JPEG
archivo.mp3	Música codificada en formato de audio MPEG capa 3
archivo.mpg	Película codificada con el estándar MPEG
archivo.o	Archivo objeto (producido por el compilador, no se ha enlazado todavía)
archivo.pdf	Archivo en Formato de Documento Portable
archivo.ps	Archivo de PostScript
archivo.tex	Entrada para el programa formateador TEX
archivo.txt	Archivo de texto general
archivo.zip	Archivo comprimido

Figura 4-1. Algunas extensiones de archivos comunes.

En algunos sistemas (como UNIX) las extensiones de archivo son sólo convenciones y no son impuestas por los sistemas operativos. Un archivo llamado *archivo.txt* podría ser algún tipo de archivo de texto, pero ese nombre es más un recordatorio para el propietario que un medio para transportar información a la computadora. Por otro lado, un compilador de C podría insistir que los archivos que va a compilar terminen con *.c* y podría rehusarse a compilarlos si no tienen esa terminación.

Las convenciones como ésta son especialmente útiles cuando el mismo programa puede manejar diferentes tipos de archivos. Por ejemplo, el compilador C puede recibir una lista de varios archivos para compilarlos y enlazarlos, algunos de ellos archivos de C y otros archivos de lenguaje ensamblador. Entonces, la extensión se vuelve esencial para que el compilador sepa cuáles son archivos de C, cuáles son archivos de lenguaje ensamblador y cuáles son archivos de otro tipo.

Por el contrario, Windows está consciente de las extensiones y les asigna significado. Los usuarios (o procesos) pueden registrar extensiones con el sistema operativo y especificar para cada una cuál programa "posee" esa extensión. Cuando un usuario hace doble clic sobre un nombre de archivo, el programa asignado a su extensión de archivo se inicia con el archivo como parámetro. Por

ejemplo, al hacer doble clic en *archivo.doc* se inicia Microsoft *Word* con *archivo.doc* como el archivo inicial a editar.

4.1.2 Estructura de archivos

Los archivos se pueden estructurar en una de varias formas. Tres posibilidades comunes se describen en la figura 4-2. El archivo en la figura 4-2(a) es una secuencia de bytes sin estructura: el sistema operativo no sabe, ni le importa, qué hay en el archivo. Todo lo que ve son bytes. Cualquier significado debe ser impuesto por los programas a nivel usuario. Tanto UNIX como Windows utilizan esta metodología.

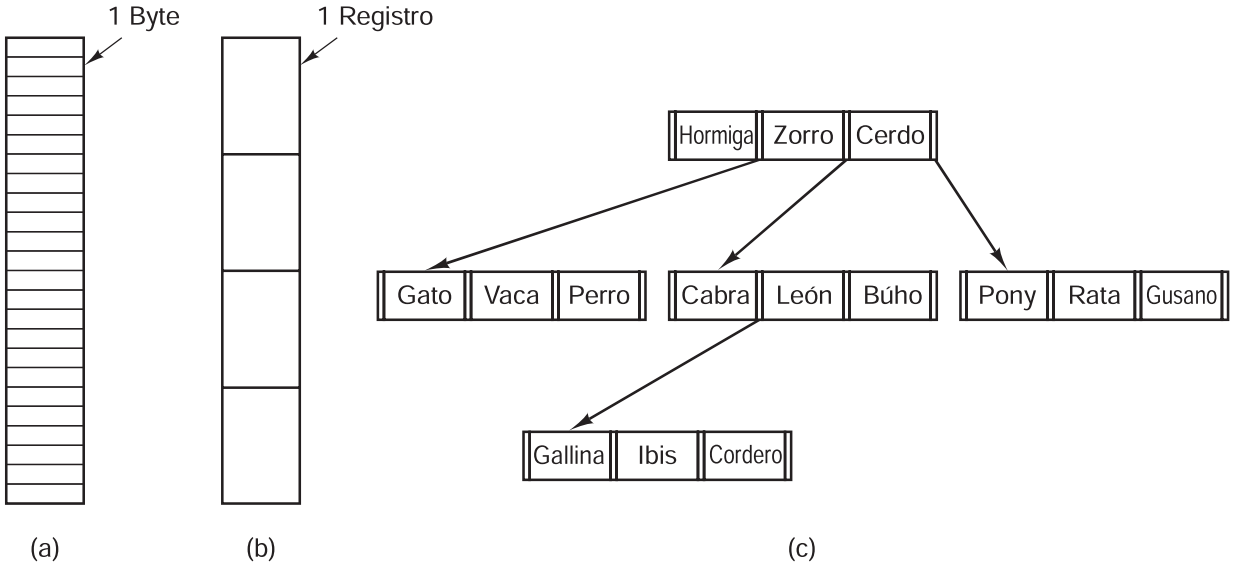


Figura 4-2. Tres tipos de archivos. (a) Secuencia de bytes. (b) Secuencia de registros. (c) Árbol.

Hacer que el sistema operativo considere los archivos sólo como secuencias de bytes provee la máxima flexibilidad. Los programas de usuario pueden colocar cualquier cosa que quieran en sus archivos y denominarlos de cualquier manera conveniente. El sistema operativo no ayuda, pero tampoco estorba. Para los usuarios que desean realizar cosas inusuales, esto último puede ser muy importante. Todas las versiones de UNIX, MS-DOS y Windows utilizan este modelo de archivos.

La primera configuración en la estructura se muestra en la figura 4-2(b). En este modelo, un archivo es una secuencia de registros de longitud fija, cada uno con cierta estructura interna. El concepto central para la idea de que un archivo sea una secuencia de registros es la idea de que la operación de lectura devuelva un registro y la operación de escritura sobrescriba o agregue un registro. Como nota histórica, hace algunas décadas, cuando reinaba la tarjeta perforada de 80 columnas, muchos sistemas operativos de mainframes basaban sus sistemas de archivos en archivos consistentes de registros de 80 caracteres; es decir, en imágenes de la tarjeta. Estos sistemas también admitían archivos con registros de 132 caracteres, que fueron destinados para la impresora de

línea (que en esos días eran grandes impresoras de cadena con 132 columnas). Los programas leían la entrada en unidades de 80 caracteres y la escribían en unidades de 132 caracteres, aunque los últimos 52 podían ser espacios, desde luego. Ningún sistema de propósito general de la actualidad utiliza ya este modelo como su sistema de archivos primario, pero en aquellos días de las tarjetas perforadas de 80 columnas y del papel de impresora de línea de 132 caracteres, éste era un modelo común en las computadoras mainframe.

El tercer tipo de estructura de archivo se muestra en la figura 4-2(c). En esta organización, un archivo consiste de un árbol de registros, donde no todos son necesariamente de la misma longitud; cada uno de ellos contiene un campo **llave** en una posición fija dentro del registro. El árbol se ordena con base en el campo llave para permitir una búsqueda rápida por una llave específica.

La operación básica aquí no es obtener el "siguiente" registro, aunque eso también es posible, sino obtener el registro con una llave específica. Para el archivo del zoológico de la figura 4-2(c), podríamos pedir al sistema que, por ejemplo, obtenga el registro cuya llave sea *pony*, sin preocuparnos acerca de su posición exacta en el archivo. Además, se pueden agregar nuevos registros al archivo, con el sistema operativo, y no el usuario, decidiendo dónde colocarlos. Evidentemente, este tipo de archivos es bastante distinto de los flujos de bytes sin estructura que se usan en UNIX y Windows, pero se utiliza de manera amplia en las grandes computadoras mainframe que aún se emplean en algún procesamiento de datos comerciales.

4.1.3 Tipos de archivos

Muchos sistemas operativos soportan varios tipos de archivos. Por ejemplo, UNIX y Windows tienen archivos y directorios regulares. UNIX también tiene archivos especiales de caracteres y de bloques. Los **archivos regulares** son los que contienen información del usuario. Todos los archivos de la figura 4-2 son archivos regulares. Los **directorios** son sistemas de archivos para mantener la estructura del sistema de archivos. Estudiaremos los directorios un poco más adelante. Los **archivos especiales de caracteres** se relacionan con la entrada/salida y se utilizan para modelar dispositivos de E/S en serie, tales como terminales, impresoras y redes. Los **archivos especiales de bloques** se utilizan para modelar discos. En este capítulo estaremos interesados principalmente en los archivos regulares.

Por lo general, los archivos regulares son archivos ASCII o binarios. Los archivos ASCII consisten en líneas de texto. En algunos sistemas, cada línea se termina con un carácter de retorno de carro. En otros se utiliza el carácter de avance de línea. Algunos sistemas (por ejemplo, MS-DOS) utilizan ambos. No todas las líneas necesitan ser de la misma longitud.

La gran ventaja de los archivos ASCII es que se pueden mostrar e imprimir como están, y se pueden editar con cualquier editor de texto. Además, si muchos programas utilizan archivos ASCII para entrada y salida, es fácil conectar la salida de un programa con la entrada de otro, como en las canalizaciones de shell. (La plomería entre procesos no es más fácil, pero la interpretación de la información lo es si una convención estándar, tal como ASCII, se utiliza para expresarla).

Otros archivos son binarios, lo cual sólo significa que no son archivos ASCII. Al listarlos en la impresora aparece un listado incomprensible de caracteres. Por lo general tienen cierta estructura interna conocida para los programas que los utilizan.

Por ejemplo, en la figura 4-3(a) vemos un archivo binario ejecutable simple tomado de una de las primeras versiones de UNIX. Aunque técnicamente el archivo es sólo una secuencia de bytes, el sistema operativo sólo ejecutará un archivo si tiene el formato apropiado. Este archivo tiene cinco secciones: encabezado, texto, datos, bits de reubicación y tabla de símbolos. El encabezado empieza con un supuesto **número mágico**, que identifica al archivo como un archivo ejecutable (para evitar la ejecución accidental de un archivo que no tenga este formato). Después vienen los tamaños de las diversas partes del archivo, la dirección en la que empieza la ejecución y algunos bits de bandera. Después del encabezado están el texto y los datos del programa en sí. Éstos se cargan en memoria y se reubican usando los bits de reubicación. La tabla de símbolos se utiliza para depurar.

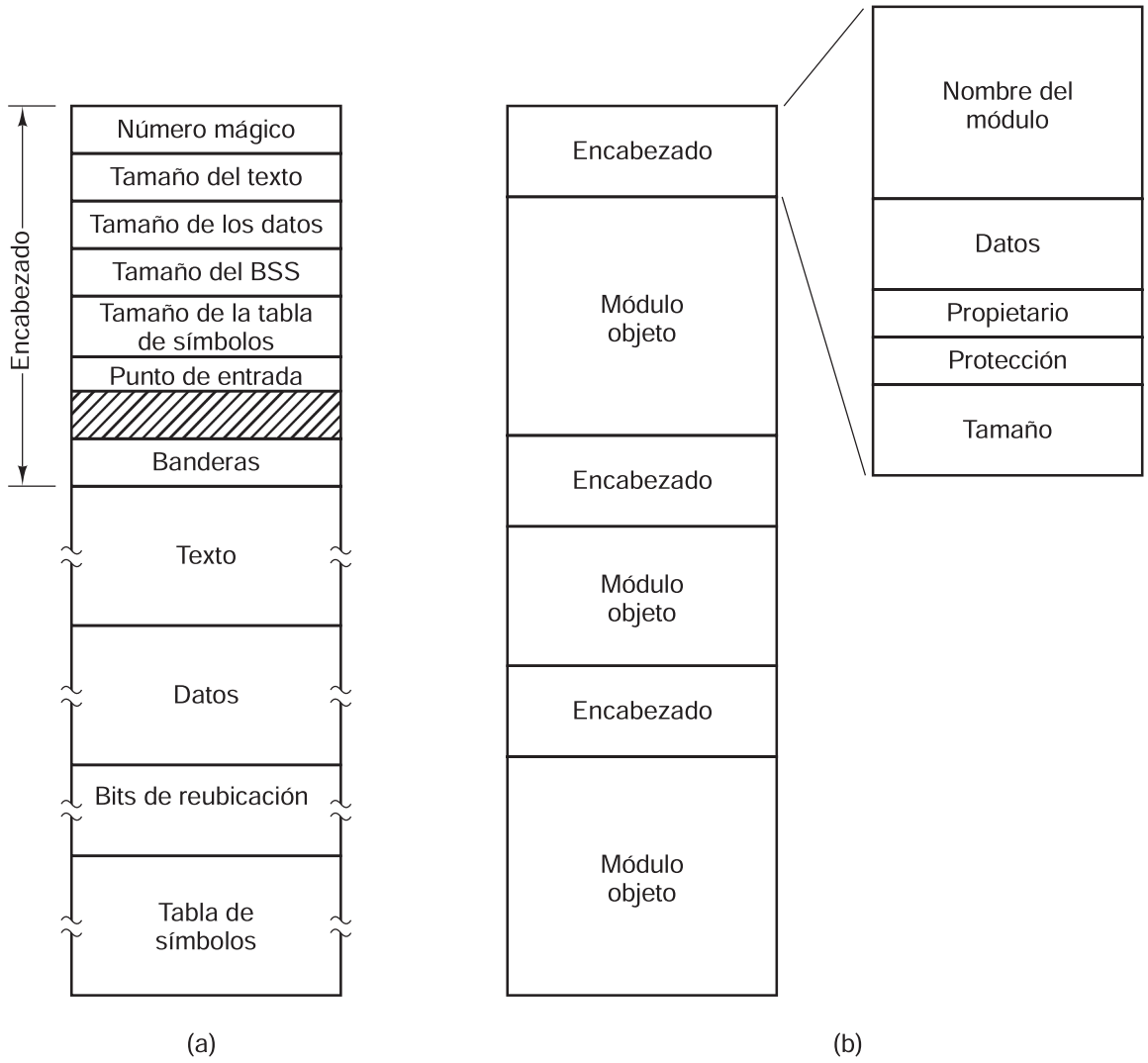


Figura 4-3. (a) Un archivo ejecutable. (b) Un archivo.

Nuestro segundo ejemplo de un archivo binario es un archivo, también de UNIX. Consiste en una colección de procedimientos (módulos) de biblioteca compilados, pero no enlazados. A cada uno se le antepone un encabezado que indica su nombre, fecha de creación, propietario, código de

protección y tamaño. Al igual que en el caso del archivo ejecutable, los encabezados de los módulos están llenos de números binarios. Al copiarlos a la impresora se produciría basura como salida.

Cada sistema operativo debe reconocer por lo menos un tipo de archivo —su propio archivo ejecutable— y algunos reconocen más. El antiguo sistema TOPS-20 (para el DECsystem 20) hacía algo más, ya que examinaba la hora de creación de cualquier archivo a ejecutar. Después localizaba el archivo de código fuente y veía si éste se había modificado desde la última vez que se creó el binario. Si así era, recompilaba automáticamente el código fuente. En términos de UNIX, el programa *make* había sido integrado al shell. Las extensiones de archivo eran obligatorias, por lo que el sistema operativo podía saber cuál programa binario se derivaba de cuál fuente.

Tener archivos fuertemente tipificados como éstos ocasiona problemas cada vez que el usuario hace algo que los diseñadores del sistema no esperaban. Por ejemplo, considere un sistema en el que los archivos de salida de un programa tienen la extensión *.dat* (archivos de datos). Si un usuario escribe un programa formateador que lea un archivo *.c* (programa de C), lo transforma (por ejemplo, convirtiéndolo a un esquema de sangría estándar) y después escribe el archivo transformado como salida, el archivo de salida será de tipo *.dat*. Si el usuario trata de ofrecer esto al compilador de C para que lo compile, el sistema se rehusará debido a que tienen la extensión incorrecta. Los intentos de copiar *archivo.dat* a *archivo.c* serán rechazados por el sistema como inválidos (para proteger al usuario contra los errores).

Mientras que este tipo de "amabilidad con el usuario" puede ayudar a los novatos, vuelve locos a los usuarios experimentados debido a que tienen que dedicar un esfuerzo considerable para sortear la idea del sistema operativo en cuanto a lo que es razonable y lo que no.

4.1.4 Acceso a archivos

Los primeros sistemas operativos proporcionaban sólo un tipo de acceso: **acceso secuencial**. En estos sistemas, un proceso podía leer todos los bytes o registros en un archivo en orden, empezando desde el principio, pero no podía saltar algunos y leerlos fuera de orden. Sin embargo, los archivos secuenciales podían rebobinarse para poder leerlos todas las veces que fuera necesario. Los archivos secuenciales eran convenientes cuando el medio de almacenamiento era cinta magnética en vez de disco.

Cuando se empezó a usar discos para almacenar archivos, se hizo posible leer los bytes o registros de un archivo fuera de orden, pudiendo acceder a los registros por llave en vez de posición. Los archivos cuyos bytes o registros se pueden leer en cualquier orden se llaman **archivos de acceso aleatorio**. Son requeridos por muchas aplicaciones.

Los archivos de acceso aleatorio son esenciales para muchas aplicaciones, como los sistemas de bases de datos. Si el cliente de una aerolínea llama y desea reservar un asiento en un vuelo específico, el programa de reservación debe poder tener acceso al registro para ese vuelo sin tener que leer primero los miles de registros de otros vuelos.

Es posible utilizar dos métodos para especificar dónde se debe empezar a leer. En el primero, cada operación `read` da la posición en el archivo en la que se va a empezar a leer. En el segundo se provee una operación especial (`seek`) para establecer la posición actual. Después de una operación `seek`, el archivo se puede leer de manera secuencial desde la posición actual. Este último método se utiliza en UNIX y Windows.

4.1.5 Atributos de archivos

Todo archivo tiene un nombre y sus datos. Además, todos los sistemas operativos asocian otra información con cada archivo; por ejemplo, la fecha y hora de la última modificación del archivo y su tamaño. A estos elementos adicionales les llamaremos **atributos** del archivo. Algunas personas los llaman **metadatos**. La lista de atributos varía considerablemente de un sistema a otro. La tabla de la figura 4-4 muestra algunas de las posibilidades, pero existen otras. Ningún sistema existente tiene todos, pero cada uno de ellos está presente en algún sistema.

Atributo	Significado
Protección	Quién puede acceso al archivo y en qué forma
Contraseña	Contraseña necesaria para acceder al archivo
Creador	ID de la persona que creó el archivo
Propietario	El propietario actual
Bandera de sólo lectura	0 para lectura/escritura; 1 para sólo lectura
Bandera oculto	0 para normal; 1 para que no aparezca en los listados
Bandera del sistema	0 para archivos normales; 1 para archivo del sistema
Bandera de archivo	0 si ha sido respaldado; 1 si necesita respaldarse
Bandera ASCII/binario	0 para archivo ASCII; 1 para archivo binario
Bandera de acceso aleatorio	0 para sólo acceso secuencial; 1 para acceso aleatorio
Bandera temporal	0 para normal; 1 para eliminar archivo al salir del proceso
Banderas de bloqueo	0 para desbloqueado; distinto de cero para bloqueado
Longitud de registro	Número de bytes en un registro
Posición de la llave	Desplazamiento de la llave dentro de cada registro
Longitud de la llave	Número de bytes en el campo llave
Hora de creación	Fecha y hora en que se creó el archivo
Hora del último acceso	Fecha y hora en que se accedió al archivo por última vez
Hora de la última modificación	Fecha y hora en que se modificó por última vez el archivo
Tamaño actual	Número de bytes en el archivo
Tamaño máximo	Número de bytes hasta donde puede crecer el archivo

Figura 4-4. Algunos posibles atributos de archivos.

Los primeros cuatro atributos se relacionan con la protección del archivo e indican quién puede acceder a él y quién no. Todos los tipos de esquemas son posibles, algunos de los cuales estudiaremos más adelante. En algunos sistemas, el usuario debe presentar una contraseña para acceder a un archivo, en cuyo caso la contraseña debe ser uno de los atributos.

Las banderas son bits o campos cortos que controlan o habilitan cierta propiedad específica. Por ejemplo, los archivos ocultos no aparecen en los listados de todos los archivos. La bandera de archivo es un bit que lleva el registro de si el archivo se ha respaldado recientemente. El programa

de respaldo lo desactiva y el sistema operativo lo activa cada vez que se modifica un archivo. De esta forma, el programa de respaldo puede indicar qué archivos necesitan respaldarse. La bandera temporal permite marcar un archivo para la eliminación automática cuando el proceso que lo creó termina.

Los campos longitud de registro, posición de llave y longitud de llave sólo están presentes en los archivos en cuyos registros se pueden realizar búsquedas mediante el uso de una llave. Ellos proporcionan la información requerida para buscar las llaves.

Los diversos tiempos llevan la cuenta de cuándo se creó el archivo, su acceso y su modificación más recientes. Éstos son útiles para una variedad de propósitos. Por ejemplo, un archivo de código fuente que se ha modificado después de la creación del archivo de código objeto correspondiente necesita volver a compilarse. Estos campos proporcionan la información necesaria.

El tamaño actual indica qué tan grande es el archivo en el presente. Algunos sistemas operativos de computadoras mainframe antiguas requieren que se especifique el tamaño máximo a la hora de crear el archivo, para poder permitir que el sistema operativo reserve la cantidad máxima de almacenamiento de antemano. Los sistemas operativos de estaciones de trabajo y computadoras personales son lo bastante inteligentes como para arreglárselas sin esta característica.

4.1.6 Operaciones de archivos

Los archivos existen para almacenar información y permitir que se recupere posteriormente. Distintos sistemas proveen diferentes operaciones para permitir el almacenamiento y la recuperación. A continuación se muestra un análisis de las llamadas al sistema más comunes relacionadas con los archivos.

1. `Create`. El archivo se crea sin datos. El propósito de la llamada es anunciar la llegada del archivo y establecer algunos de sus atributos.

2. `Delete`. Cuando el archivo ya no se necesita, se tiene que eliminar para liberar espacio en el disco. Siempre hay una llamada al sistema para este propósito.

3. `Open`. Antes de usar un archivo, un proceso debe abrirlo. El propósito de la llamada a `open` es permitir que el sistema lleve los atributos y la lista de direcciones de disco a memoria principal para tener un acceso rápido a estos datos en llamadas posteriores.

4. `Close`. Cuando terminan todos los accesos, los atributos y las direcciones de disco ya no son necesarias, por lo que el archivo se debe cerrar para liberar espacio en la tabla interna. Muchos sistemas fomentan esto al imponer un número máximo de archivos abiertos en los proceso. Un disco se escribe en bloques y al cerrar un archivo se obliga a escribir el último bloque del archivo, incluso aunque ese bloque no esté lleno todavía.

5. `Read`. Los datos se leen del archivo. Por lo general, los bytes provienen de la posición actual. El llamador debe especificar cuántos datos se necesitan y también debe proporcionar un búfer para colocarlos.

6. `Write`. Los datos se escriben en el archivo otra vez, por lo general en la posición actual. Si la posición actual es al final del archivo, aumenta su tamaño. Si la posición actual está en medio del archivo, los datos existentes se sobrescriben y se pierden para siempre.
7. `Append`. Esta llamada es una forma restringida de `write`. Sólo puede agregar datos al final del archivo. Los sistemas que proveen un conjunto mínimo de llamadas al sistema por lo general no tienen `append`; otros muchos sistemas proveen varias formas de realizar la misma acción y algunas veces ésos tienen `append`.
8. `Seek`. Para los archivos de acceso aleatorio, se necesita un método para especificar de dónde se van a tomar los datos. Una aproximación común es una llamada al sistema de nombre `seek`, la cual reposiciona el apuntador del archivo en una posición específica del archivo. Una vez que se completa esta llamada, se pueden leer o escribir datos en esa posición.
9. `Get attributes`. A menudo, los procesos necesitan leer los atributos de un archivo para realizar su trabajo. Por ejemplo, el programa *make* de UNIX se utiliza con frecuencia para administrar proyectos de desarrollo de software que consisten en muchos archivos fuente. Cuando se llama a *make*, este programa examina los tiempos de modificación de todos los archivos fuente y objeto, con los que calcula el mínimo número de compilaciones requeridas para tener todo actualizado. Para hacer su trabajo, debe analizar los atributos, a saber, los tiempos de modificación.
10. `Set attributes`. Algunos de los atributos puede establecerlos el usuario y se pueden modificar después de haber creado el archivo. Esta llamada al sistema hace eso posible. La información del modo de protección es un ejemplo obvio. La mayoría de las banderas también caen en esta categoría.
11. `Rename`. Con frecuencia ocurre que un usuario necesita cambiar el nombre de un archivo existente. Esta llamada al sistema lo hace posible. No siempre es estrictamente necesaria, debido a que el archivo por lo general se puede copiar en un nuevo archivo con el nuevo nombre, eliminando después el archivo anterior.

4.1.7 Un programa de ejemplo que utiliza llamadas al sistema de archivos

En esta sección examinaremos un programa simple de UNIX que copia un archivo de su archivo fuente a un archivo destino. Su listado se muestra en la figura 4-5. El programa tiene una mínima funcionalidad y un reporte de errores aún peor, pero nos da una idea razonable acerca de cómo funcionan algunas de las llamadas al sistema relacionadas con archivos.

Por ejemplo, el programa *copyfile* se puede llamar mediante la línea de comandos

```
copyfile abc xyz
```

para copiar el archivo *abc* a *xyz*. Si *xyz* ya existe, se sobrescribirá. En caso contrario, se creará.

```
/* Programa para copiar archivos. La verificación y el reporte de errores son mínimos. */

#include <sys/types.h>                  /* incluye los archivos de encabezado necesarios *
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]);       /* prototipo ANSI */

#define TAM_BUF 4096                    /* usa un tamaño de búfer de 4096 bytes */
#define MODO_SALIDA 0700                /* bits de protección para el archivo de salida */

int main(int argc, char *argv[])
{
        int ent_da, sal_da, leer_cuenta, escribir_cuenta:
        char bufer[TAM_BUF];

        if (argc != 3) exit(1);                 /* error de sintaxis si argc no es 3 */

        /* Abre el archivo de entrada y crea el archivo de salida */
        ent_da = open(argv[1], O_RDONLY);       /* abre el archivo fuente */
        if (ent_da < 0) exit(2);                /* si no se puede abrir, termina */
        sal_da = creat(argv[2], MODO_SALIDA);   /* crea el archivo de destino */
        if (sal_da < 0) exit(3);                /* si no se puede crear, termina */

        /* Ciclo de copia */
        while (TRUE) {
                leer_cuenta = read(ent_da,bufer,TAM_BUF); /* lee un bloque de datos */
        if (lee_cuenta <= 0) break;             /* si llega al fin de archivo o hay un error, sale del ciclo */
                escribe_cuenta = write(sal_da,bufer,lee_cuenta); /* escribe los datos */
                if (escribe_cuenta <= 0) exit(4);       /* escribe_cuenta <= 0 es un error */
        }

        /* Cierra los archivos */
        close(ent_da);
        close(sal_da);
        if (lee_cuenta==0)                      /* no hubo error en la última lectura */
                exit(0);
        else
                exit(5);                        /* hubo error en la última lectura */
}
```

Figura 4-5. Un programa simple para copiar un archivo.

El programa debe llamarse con exactamente dos argumentos, ambos nombres de archivo válidos. El primero es el archivo fuente, el segundo es el de salida.

Las cuatro instrucciones *#include* iniciales hacen que se incluya una gran cantidad de definiciones y prototipos de funciones en el programa. Esto es necesario para hacer el programa conforme a los estándares internacionales relevantes, pero no nos ocuparemos más de ello. La siguiente

línea es un prototipo de función para *main*, algo requerido por ANSI C, pero que tampoco es importante para nuestros fines.

La primera instrucción *#define* es una definición de macro que define la cadena de caracteres *TAM_BUF* como una macro que se expande en el número 4096. El programa leerá y escribirá en trozos de 4096 bytes. Se considera una buena práctica de programación dar nombres a las constantes como ésta y utilizar los nombres en vez de las constantes. Esta convención no sólo facilita que los programas sean fáciles de leer, sino también su mantenimiento. La segunda instrucción *#define* determina quién puede acceder al archivo de salida.

El programa principal se llama *main* y tiene dos argumentos: *argc* y *argv*. El sistema operativo suministra estos argumentos cuando se hace una llamada al programa. El primero indica cuántas cadenas estaban presentes en la línea de comandos que invocó al programa, incluyendo su nombre. Debe ser 3. El segundo es un arreglo de apuntadores a los argumentos. En la llamada de ejemplo anterior, los elementos de este arreglo contienen apuntadores a los siguientes valores:

```
argv[0] = "copyfile"
argv[1] = "abc"
argv[2] = "xyz"
```

Es mediante este arreglo que el programa tiene acceso a sus argumentos.

Se declaran cinco variables. Las primeras dos, *ent_da* y *sal_da*, contienen los **descriptores de archivos**: pequeños enteros que se devuelven cuando se abre un archivo. Los siguientes dos, *leer_cuenta* y *escribir_cuenta*, son las cuentas de bytes que devuelven las llamadas al sistema `read` y `write`, respectivamente. La última variable, *bufer*, es el búfer que se utiliza para guardar los datos leídos y suministrar los datos que se van a escribir.

La primera instrucción verifica *argc* para ver si es 3. Si no, termina con el código de estado 1. Cualquier código de estado distinto de 0 indica que ocurrió un error. El código de estado es el único modo de reportar errores en este programa. Una versión de producción por lo general también imprime mensajes de error.

Después tratamos de abrir el archivo fuente y crear el archivo de destino. Si el archivo fuente se abre con éxito, el sistema asigna un pequeño entero a *ent_da*, para identificar el archivo. Las llamadas siguientes deben incluir este entero, de manera que el sistema sepa qué archivo desea. De manera similar, si el archivo de destino se crea satisfactoriamente, *sal_da* recibe un valor para identificarlo. El segundo argumento para *creat* establece el modo de protección. Si falla la apertura o la creación de los archivos, el descriptor de archivo correspondiente se establece en -1 y el programa termina con un código de error.

Ahora viene el ciclo de copia. Empieza tratando de leer 4 KB de datos al *bufer*. Para ello hace una llamada al procedimiento de biblioteca *read*, que en realidad invoca la llamada al sistema `read`. El primer parámetro identifica al archivo, el segundo proporciona el búfer y el tercero indica cuántos bytes se deben leer. El valor asignado a *leer_cuenta* indica el número de bytes leídos. Por lo general este número es 4096, excepto si en el archivo quedan menos bytes. Cuando se haya alcanzado el fin del archivo, será 0. Si alguna vez *leer_cuenta* es cero o negativo, el proceso de copia no puede continuar, ejecutándose la instrucción *break* para terminar el ciclo (que de otra manera no tendría fin).

La llamada a *write* envía el búfer al archivo de destino. El primer parámetro identifica al archivo, el segundo da el bufer y el tercero indica cuántos bytes se deben escribir, en forma análoga a *read*. Observe que la cuenta de bytes es el número de bytes leídos, no *TAM_BUF*. Este punto es importante, debido a que la última lectura no devolverá 4096 a menos que el archivo sea un múltiplo de 4 KB.

Cuando se haya procesado todo el archivo, la primera llamada más allá del final del archivo regresará 0 a *leer_cuenta*, lo cual hará que salga del ciclo. En este punto se cierran los dos archivos y el programa termina con un estado que indica la terminación normal.

Aunque las llamadas al sistema de Windows son diferentes de las de UNIX, la estructura general de un programa de Windows de línea de comandos para copiar un archivo es moderadamente similar al de la figura 4-5. En el capítulo 11 examinaremos las llamadas a Windows Vista.

4.2 DIRECTORIOS

Para llevar el registro de los archivos, los sistemas de archivos por lo general tienen **directorios** o **carpetas**, que en muchos sistemas son también archivos. En esta sección hablaremos sobre los directorios, su organización, sus propiedades y las operaciones que pueden realizarse con ellos.

4.2.1 Sistemas de directorios de un solo nivel

La forma más simple de un sistema de directorios es tener un directorio que contenga todos los archivos. Algunas veces se le llama **directorio raíz**, pero como es el único, el nombre no importa mucho. En las primeras computadoras personales, este sistema era común, en parte debido a que sólo había un usuario. Como dato interesante, la primera supercomputadora del mundo (CDC 6600) también tenía un solo directorio para todos los archivos, incluso cuando era utilizada por muchos usuarios a la vez. Esta decisión sin duda se hizo para mantener simple el diseño del software.

En la figura 4-6 se muestra un ejemplo de un sistema con un directorio. Aquí el directorio contiene cuatro archivos. Las ventajas de este esquema son su simpleza y la habilidad de localizar archivos con rapidez; después de todo, sólo hay un lugar en dónde buscar. A menudo se utiliza en dispositivos incrustados simples como teléfonos, cámaras digitales y algunos reproductores de música portátiles.

4.2.2 Sistemas de directorios jerárquicos

Tener un solo nivel es adecuado para aplicaciones dedicadas simples (e incluso se utilizaba en las primeras computadoras personales), pero para los usuarios modernos con miles de archivos, sería imposible encontrar algo si todos los archivos estuvieran en un solo directorio.